Pixels to Power Pellets: Reward Shaping, Input Modality and Architecture in Pixel-Based PPO for Ms. Pac-Man

[Author Name 1]
[sunet1]@stanford.edu

[Author Name 2]
[sunet2]@stanford.edu

Key Information to Include

- **Team contributions:** [Member 1: PPO/DQN library, environment wrappers, reward shaping, penalty experiments. Member 2: LSTM and ViT networks, RGB-from-gray conversion, evaluation, analysis and writing.] *(Replace with the real split, 1–2 sentences per person.)*
- **Project type:** Custom project.
- **Mentor:** [CS224N staff mentor / external mentor name]
- **External collaborators:** None. **Shared project:** No.
- **Code:** `pacman_rl` (PPO, DQN, vectorised environments, networks) and the notebooks that call it were written by us; the emulator and environment come from Gymnasium/ALE (see §3).

Abstract

Reinforcement-learning agents that play Atari from raw pixels are a standard testbed for deep policy-gradient methods, but a plain Proximal Policy Optimization (PPO) agent on Ms. Pac-Man quickly stalls at roughly 700 points, about 2.5× a random policy. We try to find out why and what moves it past that plateau. We implement PPO from scratch in PyTorch and run a chain of hypothesis-driven experiments: (H1) the agent idles because dying is cheap and waiting is free, so we add a death penalty (*dp*) and a per-step penalty (*sp*); (H2) grayscale frames erase the colour of frightened ghosts, so we try RGB input and a gray-to-RGB weight-transfer scheme; (H3) a single frame lacks memory, so we compare a CNN with a CNN+LSTM and a small Vision Transformer (ViT). Over 23 saved training runs of 1–24 M agent steps (about 84 M steps in total), we find that (i) RGB input gives no measurable gain at 1–3 M steps; (ii) a large death penalty from the start hurts (499 vs. 651 points, two seeds); (iii) long, low-learning-rate fine-tuning with a small step penalty lifts the ViT to a training score of 1125 (median 975; best game 4720), driven by roughly doubling ghost captures per game; (iv) penalties that are too large make the agent collapse, and the value-loss scale gives an early warning of this. Our evaluation also shows 100–200 points of run-to-run noise on the same model, so many small differences cannot be interpreted. All figures are regenerated from the saved logs by a script in the project repository.

1 Introduction

Deep reinforcement learning (RL) from pixels, introduced for Atari by Mnih et al. [15], remains a central benchmark for learning from high-dimensional observations. Policy-gradient methods such as PPO [20] are popular because they are simple and robust, yet their behaviour in a specific game depends strongly on details that papers rarely foreground: reward scaling, how episodes are terminated, and how input frames are encoded [6, 8, 11].

Ms. Pac-Man is a good case study. The reward is sparse in the sense that matters: ordinary dots give 10 points, but the large rewards (ghosts worth 200–1600 points) require a *sequence* of decisions: eat a power pellet, notice that the ghosts have turned blue, chase them before the effect ends. Ghost movement is also stochastic, so

there is no fixed route to memorise. When we trained a standard CNN-based PPO agent we saw the following: the score jumped within the first $\sim$25 k steps, then stayed near 700 even when training was extended from 1 M to 2 M steps (random play scores 200–280). Watching the agent suggested it either stood still or repeated a fixed pattern, and that after eating a power pellet it did not hunt ghosts.

This report documents the diagnosis. We treat each suspected cause as a hypothesis and design the cheapest experiment that could confirm or reject it: reward shaping for idling (H1), RGB input for the lost ghost colour (H2), and recurrent or attention-based encoders for missing memory (H3). The contribution is *not* a new algorithm, but (1) an end-to-end from-scratch PPO implementation with LSTM and ViT policies and a weight-transfer scheme from gray to RGB; (2) a controlled comparison of penalty schedules, input modality and architecture, with their negative results; (3) a quantified failure mode, penalty-induced collapse, together with a cheap diagnostic (the value loss); and (4) an honest look at evaluation noise, which affects how much of the RL literature's small gaps can be believed. Our best agent (a ViT fine-tuned over 13 M steps) reaches a mean training score of 1125, roughly four times random but far below human-level play.

2 Related Work

Deep RL on Atari. DQN [14, 15] established learning from stacked 84×84 gray frames with reward clipping, and Rainbow [9] combined later improvements. The Arcade Learning Environment [3] and its evaluation protocol recommendations, including sticky actions [13], define the benchmark. Actor-critic methods [16] and PPO [20] with generalised advantage estimation [19] are the main on-policy alternatives. We use PPO because it parallelises easily on a single laptop GPU. Previous work on Ms. Pac-Man specifically decomposes the reward into per-object value functions [24]; we instead keep a single scalar reward and ask which cheaper modifications help.

Implementation details and reproducibility. Engstrom et al. [6] and Huang et al. [11] show that code-level choices (reward normalisation, learning-rate annealing, value clipping) can matter as much as the algorithm, while Henderson et al. [8] and Agarwal et al. [1] document how few seeds and noisy evaluation produce unreliable conclusions. These works motivate our decision to report evaluation noise explicitly and to flag every comparison that has only one seed.

Reward shaping. Potential-based shaping preserves optimal policies [17], but a constant per-step cost or a death penalty is *not* potential-based and can change the optimum. Reward misspecification is a known safety concern [2, 21]. Our collapse results (§5) are a concrete small-scale instance.

Memory and attention in RL. Recurrent Q-networks [7] use an LSTM [10] to handle partial observability. Transformers [25] and Vision Transformers [5] are attractive encoders, but plain Transformers are known to be hard to train with RL objectives, which motivated stabilising variants [18]. Because CS224N focuses on sequence models, we include both an LSTM and a ViT whose patch tokens are processed with self-attention, and compare them with the Nature-DQN CNN.

3 Approach

Vocabulary. A *step* is one agent decision (4 emulator frames). A *life* ends when Pac-Man is caught; a *game* is up to three lives. SCORE always means the real game score summed over a game (never the shaped reward). *Training score* is the average SCORE of games finished during training (stochastic, drifting policy); *evaluation score* is measured on separate games with a fixed seed set.

3.1 Environment and reward

We use `ALE/MsPacman-v5` from Gymnasium [23] (code not ours): 9 discrete actions, frame-skip 4, no sticky actions. The observation is the last four frames, resized to 84×84, stacked as $(4, 84, 84)$ for gray input or $(12, 84, 84)$ for RGB. During training, losing a life is reported as episode termination ("episodic life") while the underlying game continues, so the value function sees shorter, well-defined episodes. For evaluation this wrapper is off and one full game is played (at most 4500 steps).

The learning reward is our own variant of reward compression (replacing clipping):

$$\tilde{r}_t = \ln\bigl(1 + r_t/10\bigr) \;-\; dp \cdot \mathbb{1}[\text{life lost at } t] \;-\; sp, \tag{1}$$

where r_t is the true game reward. A dot (10) gives 0.69, a power pellet (50) gives 1.79, and the four successive ghosts (200/400/800/1600) give $3.04/3.71/4.39/5.08$, so large events keep a larger, but not dominant, weight than clipping would allow. *dp* (death penalty) and *sp* (per-step cost) implement H1.

3.2 PPO

We implemented PPO from scratch in PyTorch [20]: N=32 parallel environments × 512 steps per rollout (16 384 samples per update), GAE with γ=0.99, λ=0.95 [19], and the clipped objective

$$L = \mathbb{E}_t\Bigl[\min\bigl(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1{-}\epsilon, 1{+}\epsilon)\hat{A}_t\bigr)\Bigr] - c_1\,\mathbb{E}_t\bigl[(V_\theta(s_t) - \hat{R}_t)^2\bigr] + c_2\,\mathbb{E}_t\bigl[\mathcal{H}(\pi_\theta(\cdot|s_t))\bigr], \tag{2}$$

with $\rho_t = \pi_\theta/\pi_{\theta_{\text{old}}}$, ϵ=0.1, c_1=0.5, c_2=0.01, 4 epochs, 4 minibatches, gradient-norm clip 0.5, Adam [12] with linearly decayed learning rate (full list in Appendix A). Training used one RTX 3070 Ti laptop GPU.

3.3 Policies

All networks share an actor head (9 logits) and a critic head (scalar).

- **CNN (baseline).** Nature-DQN trunk [15]: Conv(8×8, s4)–Conv(4×4, s2)–Conv(3×3, s1)–FC-512.
- **CNN+LSTM.** Same trunk plus an LSTM (hidden 512). The hidden state is reset when a life is lost. Because the recurrent state must flow through a whole rollout, minibatches are formed by splitting environments (each keeps its 512-step sequence) rather than shuffling individual steps.
- **ViT.** A small Vision Transformer [5]: 12×12 patches (7×7=49 tokens + a CLS token), width 128, depth 4, 4 heads, MLP ratio 2, pre-norm blocks [26], built on `torch.nn.TransformerEncoder`.
- **Gray-to-RGB weight transfer (ours).** To test colour without discarding what a gray agent has learned, we expand the first conv layer from 4 to 12 input channels by setting each frame's R, G, B weights to the gray weight $\times(0.299, 0.587, 0.114)$. This makes the RGB network's initial output approximately equal to the gray network's (the ITU-R 601 luminance mix), and all other layers are copied unchanged.

3.4 Experimental protocol

The study proceeded in five phases, each motivated by what the previous phase showed (§4): **P0** baseline plateau (CNN gray, 2 M steps); **P1** penalty experiments (const-low, const-high, curriculum; 2 seeds, 1 M steps); **P2** gray vs. RGB and CNN vs. LSTM vs. ViT (1–3 M steps); **P3** long fine-tuning of LSTM and ViT (13 M steps in four stages) with a small step penalty and a lower learning rate; **P4** RGB networks initialised from gray with penalty ramps, including three step-penalty magnitudes.

4 Experiments

4.1 Data and task

There is no fixed dataset: the agent generates its own experience from the emulator. Input is the 4-frame stack above; output is a distribution over 9 joystick actions. The goal is to maximise the game score over a three-life game.

4.2 Evaluation method

We report two quantities. (a) **Training score:** mean SCORE over the last 100–300 games of a run (the exact window is stated). It averages thousands of games, but it is measured with a stochastic policy that is still being updated, so it is biased low relative to a converged policy and only suitable for trends. (b) **Evaluation score:** 20 full games on seeds 1000–1019 with the policy *sampled* (not greedy), reported as mean$\pm$std with the standard error $\approx \text{std}/\sqrt{20}$. We also track behavioural statistics from the game reward: ghosts eaten per game, power pellets per game, steps per game, and policy entropy (maximum $\ln 9 = 2.20$). Ghost and pellet counts come from exact reward matches (50 for a pellet, 200/400/800/1600 for a ghost); fruit worth 200 points from level 2 can occasionally be miscounted as a ghost. Random play scores 274 on average (5 games; other 5-game samples gave 198 and 282, so this baseline is itself noisy).

4.3 Experimental details

Unless stated, *dp*$= 2.5$, *sp*$= 0$, learning rate 2.5×10^{-4} (ViT: 10^{-4}) decayed linearly to 0, 32 environments. Fine-tuning stages (P3) restart from the previous checkpoint with learning rate 10^{-4} or 5×10^{-5} and *sp*$= 0.05$. Because each call to the trainer decays the learning rate to 0, long runs have a sawtooth schedule. Typical throughput was 240–260 agent steps/s (ViT); a 1 M-step run takes about an hour. Early in the project we found that a notebook cell had restarted from a fresh network at each call while appending logs to the same file, so what we thought was one run was in fact three concatenated runs. We fixed this and draw conclusions only from stages that we can verify.

4.4 Results

P0: the plateau. The baseline CNN (gray, 16 environments, *dp*$= 0$) reached 727 and 767 (mean of the last 100 games in two consecutive 2 M-step runs; the second started from the first). Entropy fell from 2.20 to ≈ 1.2 and the mean life length stayed at 205–220 steps for the whole run. Doubling the steps from 1 M to 2 M did not change the score, which ruled out "just train longer" at this learning rate as a quick fix.

P1: penalty schedules (H1). Table 1 compares a low constant penalty, a high constant penalty from step 0 (*dp*/*sp*$= 20/0.1$) and a curriculum that ramps from $2.5/0$ to $10/0.05$ to $20/0.1$ within one run. The seed-to-seed

Schedule	Seed 0	Seed 1	Mean	Final entropy (s0/s1)
const-low (2.5/0)	703	598	**651**	1.74 / 1.79
const-high (20/0.1)	464	533	499	2.03 / 1.93
curriculum (2.5→20)	624	690	**657**	1.67 / 1.65

Table 1: Training score (last 100 games), gray CNN, 1 M steps. The high penalty from the start is $\sim$150 points worse, and learns more slowly (higher entropy); const-low and curriculum are indistinguishable.

spread inside one row (70–105) is similar to the gaps between const-low and curriculum, so only const-high

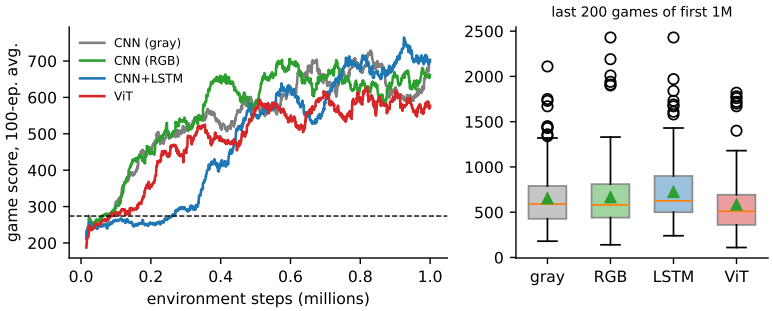


Figure 1: First 1 M steps. Left: 100-game moving average of the training score, dashed line is random play. Right: distribution of the last 200 games (triangle: mean). RGB ≈ gray; the LSTM is slightly ahead and the ViT slightly behind at this budget.

is separable. A higher penalty therefore did not solve idling, and the curriculum added nothing over the cheap setting.

P2: input modality and architecture (H2, H3). In a 3 M-step A/B test (one seed, $dp = 2.5$) the 20-game moving average fluctuated between 610 and 860 for gray and between 662 and 826 for RGB, with no consistent winner. Table 2 compares the four architectures under identical conditions for the first 1 M steps; Figure 1 shows the curves. RGB equals gray within one standard error, and the LSTM's lead (+70) is about two

	CNN (gray)	CNN (RGB)	CNN+LSTM	ViT
Training score (last 200 games of 1 M)	653 ± 23	665 ± 26	723 ± 24	583 ± 22

Table 2: Mean±standard error over the last 200 games of the first 1 M steps, $dp = 2.5$, $sp = 0$, one seed each (ViT uses lr 10^{-4}, others 2.5×10^{-4}).

standard errors with a single seed. None of these gaps are decisive at 1 M steps.

P3: long fine-tuning. With a small step penalty ($sp = 0.05$) and a lower learning rate, both sequence-aware models continued to improve rather than plateau (Figure 2). Table 3 gives the training score by stage and Table 4 the final summary. Re-scoring the ViT checkpoints during the last stage gave evaluation scores of

Stage (cumulative steps)	pre (2 M)	P1 (3 M)	A (8 M)	C (13 M)
ViT	492	655	818	**1181**
CNN+LSTM	665	729	744	967

Table 3: Training score (last 100 games of each stage). A branch that raised the death penalty to 10 for 1 M steps (stage B) scored 858 (ViT) and 732 (LSTM), i.e. no better than stage A.

1115, 1208, 1296 and 1092 (20 games each, std 450–560, medians 905–1175), consistent with the training score of ≈1100–1180.

P4: RGB from gray and penalty magnitude. Table 5 shows RGB networks initialised from a trained gray CNN (1 M steps) and trained with a ramping death penalty (2.5→5→10) under different step-penalty values. Only the setting with $sp = 2$ failed outright; its value loss was two orders of magnitude larger than in the other runs.

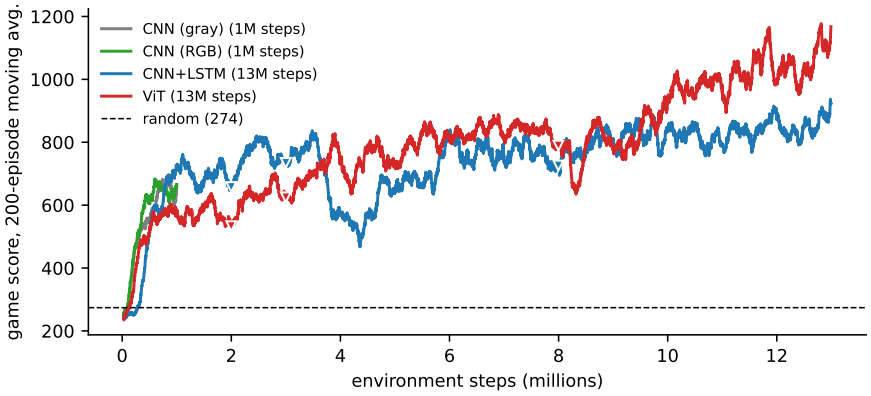


Figure 2: Training score (200-game moving average) for the best run of each model type. LSTM and ViT are chains of stages (triangles mark stage changes) of 2, 1, 5 and 5 M steps; after the first stage they use $sp = 0.05$ and a lower learning rate, so they are *not* under the same conditions as the two 1 M-step CNNs.

Model	Steps	Mean	Median	P90	Best game	Entropy
CNN (gray)	1 M	652	580	1140	2260	1.75
CNN (RGB)	1 M	640	560	1070	2460	1.77
CNN+LSTM	13 M	900	800	1391	4600	1.54
ViT	13 M	**1125**	**975**	**1940**	4720	1.76

Table 4: Last 300 training games of each model's best run (standard error of the mean: 19, 19, 25, 34). The score distribution is heavy-tailed, so the mean exceeds the median in all rows.

5 Analysis

What did the ViT learn? The headline score gain is visible in behavioural counters rather than in survival alone (Figure 3). Up to 8 M steps both ViT and LSTM sit at ≈0.6 ghosts and ≈1.0 pellets per game, the same level as the plateaued CNN. After 8 M steps the ViT rises to ≈1.06 ghosts and ≈1.58 pellets per game (last 1 M steps), while the LSTM stays at 0.60 and 0.92, and the ViT also survives about 10% longer (from ≈650 to ≈740 steps/game). Because ghosts are worth 200–1600 points, doubling ghost captures explains most of the score increase. This partly overturns our earlier reading that no model had learned to hunt ghosts, but it appears only after ∼10 M steps, and our ghost counter cannot separate fruit from ghosts (see §4).

Penalty magnitude, value loss and collapse. Penalties that are too large destroyed learning in two independent places. First, the RGB run with $sp = 2$: per-step cost 2 over a typical ≈215-step life gives ≈ -430 per life, far more than any positive reward in Eq. (1) (a dot is 0.69). Value loss was ≈300–400 versus ≈1 elsewhere, and a continuation run degraded to near-random play (steps/life ≈128, almost no pellets). Second, when the best ViT was trained further with $dp = 5$ and $sp = 0.3 \to 1 \to 0.5$ (shaded region of Figure 4), the training score fell from ≈1140 to ≈490 within one stage, and evaluation fell from 1092 to 434. Our explanation, which we have not isolated with a counterfactual run, is that with a negative per-step reward and termination on death, ending the life early becomes rational whenever dp is smaller than the penalty the agent expects to pay over the rest of the life ($\approx sp \times 200$ steps). At $sp = 1$ that is ≈200, versus $dp = 5$. This is consistent with the shorter lives (188–192 steps vs. ≈215) and with fewer pellets per life in the logs. Raising dp alone to 10 (stage B) did not collapse learning but did not help either. A practical outcome: a value-loss jump by 10× or more over a run's baseline is a cheap early warning that reward scale is wrong, and in the cases above it appeared long before the score did.

sp	Steps	Train score (1st/last 100)	Eval (20 games)	Value loss
0.05	10.0 M	664 / 646	640 ± 73	$1.3 \rightarrow 1.7$
0.3	24.0 M	604 / 779	626 ± 72	$3.5 \rightarrow 2.8$
0.5	3.0 M (stopped)	681 / 387	–	$15.3 \rightarrow 13.7$
2	7.65 M	543 / 492	510 ± 68	$394 \rightarrow 283$

Table 5: RGB-from-gray runs (one seed each). Eval is mean±standard error. The gray source model scores 608 and 828 in two separate 20-game evaluations (see §5).



Figure 3: Behaviour metrics in 0.5 M-step bins for the four best runs (dotted lines: stage changes). The ViT's gain after ∼8 M steps comes with more ghosts and pellets per game; the LSTM keeps lower policy entropy but gains little.

Does colour help? A single-frame check. After eating a power pellet, a frame in RGB contained 116 blue pixels (frightened ghost). Its luminance in gray is 109, *identical* to a common maze gray level (109), so the ghost is plausibly hard to distinguish in gray. Yet RGB did not improve the score or ghost captures: in the last 1 M steps both the gray and the RGB CNN eat 0.59 ghosts per game. We therefore cannot support H2 at this budget: the information may be recoverable from the gray image (e.g. via ghost motion), or the agent may not yet be able to exploit the signal. This is a single-frame analysis and we did not complete a direct behavioural test around frightened ghosts. A RAM-based ghost-state probe was written but not validated on Ms. Pac-Man, so we do not report it.

Evaluation noise limits what we can claim. The same gray model scored 828 in one 20-game evaluation and 608 in another; a ViT checkpoint scored 748, 660 and 499 across three evaluations with the same seed set. With std 200–500 per 20 games, the standard error is 50–100, so any gap below ∼150 points between two models is not interpretable. We suspect unseeded action sampling and trajectory-dependent environment noise but did not identify the cause. For this reason Tables 1–4 lean on thousands of training games, accepting their bias, and we describe single-seed comparisons as trends. This also tempers the architecture comparison: ViT vs. LSTM differ in learning rate, stage timing and seed count, so "ViT $>$ LSTM" is a hypothesis for future controlled testing, not a result.

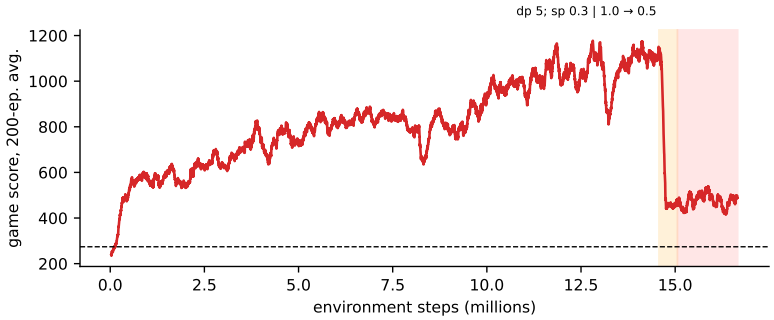


Figure 4: ViT training score over the full 16.65 M-step chain. The shaded region marks the stages where we raised penalties ($dp = 5$, $sp = 0.3$, then 1.0 and 0.5); the score drops from $\approx$1140 to $\approx$490 and does not recover.

6 Conclusion

Starting from a PPO agent stuck at $\sim$700 points, we tested reward shaping, RGB input and sequence models. The solid findings are: (1) long, low-learning-rate fine-tuning with a small step penalty let the ViT reach a training score of 1125 (best game 4720), with the main behavioural change being about twice as many ghosts eaten per game; (2) large penalties from the start (const-high) hurt, and penalties above a threshold ($sp \geq 0.3$ for the ViT, $sp = 2$ for the RGB CNN) cause collapse; (3) RGB input and a death-penalty curriculum gave no measurable benefit at 1–10 M steps; (4) evaluation noise of 100–200 points means many small gaps are not interpretable. The main limitations are one seed for most conditions, confounded changes between stages (learning rate, penalty, architecture), 20-game evaluations and a coarse ghost counter. We did not obtain DQN results on Ms. Pac-Man. Next steps are: run $\geq$3 seeds with $\geq$50 evaluation games and fixed action-sampling seeds; run a ViT/LSTM comparison under identical learning rate and step budget; repeat the 13 M-step schedule without the step penalty to separate its effect from training length; test exploration bonuses [4]; and probe frightened-ghost behaviour directly.

Ethical Considerations

Reward misspecification and what it teaches about deployment. The clearest failure in this project was a reward-design error: a per-step penalty that made quitting early the best strategy, so the agent got *worse* at the intended task while obeying the objective. The agent was only a game character, but the same mechanism (optimising a proxy that omits something important) matters when RL policies are used for resource allocation, content recommendation, or control. Our case is specific because the penalty was introduced to fix a visible symptom (idling), and the fix produced a harder-to-see failure; the problem was detected only by a diagnostic (value loss) that we happened to log. Mitigations: (i) log a small set of intended-behaviour metrics next to the shaped return (here: pellets, ghosts, steps per life) and stop training automatically when they diverge from the true objective, as we did with an early-stop threshold on training score; (ii) test every added penalty on a short run first, with value-loss and behaviour checks, before long training; and (iii) in a deployed setting, keep a human review step for any change to the reward function.

Compute cost and misleading evidence. Our results rest on 23 saved runs of up to 24 M steps each (about 84 M steps in total), run on a GPU laptop. Compute-hungry RL experiments carry an energy cost [22], and the cost is wasted when single-seed comparisons are mistaken for findings; over-confident claims can also lead other groups to repeat the same expensive experiments. Mitigations: report seeds, evaluation size and noise (as in §5); release logs and the figure-generating script so others can reuse our runs; and prefer short multi-seed pilots before long runs.

Game automation. A pixel-only agent that plays a commercial game can, with modest changes, be used as a bot. For a 1980s arcade game the harm is low, but the same pipeline could be reused for online games, where automation breaks fair play. A mitigation is to share only the training code for the emulator environment, not any interface to live online services, and to state this intended scope in the repository's documentation.

References

[1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[2] Dario Amodei, Chris Olah, Jacob Steinhardt, Paul Christiano, John Schulman, and Dan Mané. Concrete problems in ai safety. *arXiv preprint arXiv:1606.06565*, 2016.

[3] Marc G. Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47:253–279, 2013.

[4] Yuri Burda, Harrison Edwards, Amos Storkey, and Oleg Klimov. Exploration by random network distillation. In *International Conference on Learning Representations (ICLR)*, 2019.

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.

[6] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on ppo and trpo. In *International Conference on Learning Representations (ICLR)*, 2020.

[7] Matthew Hausknecht and Peter Stone. Deep recurrent q-learning for partially observable MDPs. In *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.

[8] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

[9] Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *AAAI Conference on Artificial Intelligence*, 2018.

[10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[11] Shengyi Huang, Rousslan Fernand Julien Dossa, Antonin Raffin, Anssi Kanervisto, and Weixun Wang. The 37 implementation details of proximal policy optimization. In *ICLR Blog Track*, 2022.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[13] Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

[14] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.

[15] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[16] Volodymyr Mnih, Adrià Puigdomènech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2016.

[17] Andrew Y. Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning (ICML)*, 1999.

[18] Emilio Parisotto, Francis Song, Jack Rae, Razvan Pascanu, Caglar Gulcehre, Siddhant Jayakumar, Max Jaderberg, Raphaël Lopez Kaufman, Aidan Clark, Seb Noury, et al. Stabilizing transformers for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.

[19] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.

[20] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[21] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[22] Emma Strubell, Ananya Ganesh, and Andrew McCallum. Energy and policy considerations for deep learning in NLP. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[23] Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032*, 2024.

[24] Harm van Seijen, Mehdi Fatemi, Joshua Romoff, Romain Laroche, Tavian Barnes, and Jeffrey Tsang. Hybrid reward architecture for reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[25] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[26] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tie-Yan Liu. On layer normalization in the transformer architecture. In *International Conference on Machine Learning (ICML)*, 2020.

Parameter	Value	Parameter	Value
Parallel envs $\times$ steps/rollout	32×512	Clip ϵ	0.1
Epochs / minibatches	4 / 4	c_1 (value), c_2 (entropy)	0.5, 0.01
γ, λ	0.99, 0.95	Grad-norm clip	0.5
Learning rate (from scratch)	2.5×10^{-4}	Fine-tuning learning rate	10^{-4} or 5×10^{-5}
Frame skip, sticky prob.	4, 0	Max evaluation steps	4500
ViT	patch 12, dim 128, depth 4, 4 heads	LSTM hidden	512

Table 6: PPO and network hyperparameters. Hardware: RTX 3070 Ti Laptop GPU, PyTorch 2.3.1 (CUDA 12.1), Gymnasium 0.29.1.

A Hyperparameters and reproducibility

All figures and the numbers in the text are regenerated from the saved run logs by `make_figures.py`. Two evaluations of the same model can differ by 100–200 points (§5), so re-running the 20-game evaluation will not reproduce the exact values in Table 5.